ContractGuard

Technical documentation

Architecture, module guides, testing, deployment, demo and decision log

Generated 08 November 2026 from the Markdown files in `docs/`

Contents

1. Architecture
2. Module guide
3. 01 Foundations: models, configuration, errors
4. 02 Ingestion and OCR
5. 03 Cleaning and normalisation
6. 04 Clause segmentation
7. 05 Clause classification and embeddings
8. 06 Fact and metadata extraction
9. 07 Policy engine
10. 08 LLM explanations
11. 09 Pipeline, report and CLI
12. 10 Backend API
13. 11 Streamlit UI
14. 12 Training the classifier
15. 13 Gold set and evaluation
16. 14 Tests and tooling
17. Testing strategy
18. Deployment
19. Demo script
20. Decision log

Architecture

ContractGuard follows the HLD v1.0 without changes. This file is the working summary; the HLD is the source of truth for details.

Layers

Layer	Owns	Must not
`frontend/` (Streamlit)	Upload form, progress polling, report dashboard, clause explorer, policy editor	Import `contractguard`; it talks to the API over HTTP only
`backend/` (FastAPI)	Upload validation, job queue (one job at a time), SQLite persistence, REST contracts	Contain NLP logic
`contractguard/`	Ingestion, cleaning, segmentation, classification, extraction, policy engine, explanations, report	Import FastAPI or Streamlit
`config/`, `policies/`	Every threshold, pattern, label map and rule	Live in Python code
`training/`, `evaluation/`	Offline model building and metrics	Run at request time

Stages exchange Pydantic models only (`contractguard/models.py`): `Document` → `list[Clause]` → `list[Fact]` → `list[Finding]` → `Report`.

```
flowchart LR
    UI[Streamlit UI] -->|HTTP JSON| API[FastAPI + job runner]
    API --> P[contractguard pipeline]
    subgraph P[contractguard]
      I[Ingestion] --> C[Cleaning] --> S[Segmentation] --> K[Classifier]
      K --> X[Fact extraction] --> E[Policy engine] --> L[Explainer] --> R[Report builder]
    end
    I -.-> T[[Tesseract]]
    L -.-> LLM[[Gemini / Ollama / Mock]]
    P -.-> DB[(SQLite + files)]
```

The policy engine decides every status and the policy file decides every severity. The LLM only writes explanations after that, and its quotes are checked against the clause text.

Flow 1: analyse one contract

1. **Origin.** The reviewer uploads a PDF or DOCX with contract type, our organization's name, policy id and the AI toggle.
2. **Validation (API).** Extension and magic bytes must agree (`%PDF`, ZIP `PK`); ≤ 20 MB, ≤ 60 pages, ≤ 60,000 words; filename sanitized. Failure → HTTP 400 `{error_code, message}`.
3. **Storage.** File saved to `data/uploads/{contract_id}/`; `contracts` row with status `QUEUED`; the API returns 202 with the id.
4. **Processing** (background, one at a time): ingestion (PyMuPDF per page; Tesseract only for pages with < 50 characters and an image; python-docx with Word numbering rebuilt) → cleaning → segmentation → classification (+ embedding fallback) → fact extraction → policy evaluation.
5. **Explanation.** Only NON_COMPLIANT, MISSING and NEEDS_REVIEW findings, at most 15. One clause per LLM call. The quoted evidence must be an exact substring of the clause, else retry once, then a template.

6. **Storage.** Report JSON stored whole in `contracts.report_json`; clause and finding rows for filtering; LLM answers cached in `llm_cache`.
7. **Return.** The UI polls `/status` every 2 s, then reads `/report` once.

Failure paths. Encrypted PDF or missing Tesseract → clear error. OCR confidence < 70 → report warning. LLM down or rate-limited → template explanations, report still produced. Stage exception → `FAILED` with a readable message. Ambiguous or missing fact → `NEEDS_REVIEW`, never a guess.

Flow 2: edit a policy

`PUT /policies/{id}` → JSON Schema validation (`policies/policy.schema.json`) → old file copied to `policies/history/{id}@{version}.json` → patch version bumped. Every report records the policy version it used.

Flow 3: offline training

LEDGAR and CUAD → `config/label_map.yaml` → splits (LEDGAR official; CUAD grouped by contract, 80/10/10, seed 42) → five models compared → best by validation macro-F1 saved with per-class thresholds and prototypes in `models/`. The gold set is never used for training or tuning.

Security and privacy boundaries

- The only data leaving the machine is the single clause text sent to Gemini for a finding. Ollama keeps everything local. Demos use synthetic or public contracts only.
- Secrets live in `.env` (gitignored). `contractguard config` hides the API key.
- Uploads are untrusted: magic-byte check, size/page limits, sanitized filenames.
- No authentication in V1 (single user, localhost). CORS allows the Streamlit origin only.
- Full contract text is never logged at INFO. DELETE removes the file and all rows.

Module guide

One guide per part of the code. Each guide answers the same questions in the same order:

1. **What** the module does, in one paragraph.
2. **Why** it exists and why it is built this way.
3. **How** it works, step by step.
4. **Callers and dependencies**: who calls it and what it calls.
5. **Data in and out**: the exact types that cross its boundary.
6. **Errors and edge cases**: what can go wrong and what happens then.
7. **Configuration**: the settings and pattern files that change its behaviour.
8. **Alternatives considered**: what else was possible and why it was not chosen.
9. **Interview notes**: questions an examiner is likely to ask, with short answers.

Decision numbers such as D23 refer to `docs/DECISIONS.md`. HLD section numbers refer to the high-level design document the project was built from.

#	Guide	Code
01	Foundations: models, configuration, errors	`contractguard/models.py`, `config.py`, `errors.py`
02	Ingestion and OCR	`contractguard/ingestion/`
03	Cleaning and normalisation	`contractguard/preprocessing/`
04	Clause segmentation	`contractguard/segmentation/`
05	Clause classification and embeddings	`contractguard/classification/`, `contractguard/embeddings/`
06	Fact and metadata extraction	`contractguard/extraction/`, `classification/contract_type.py`
07	Policy engine	`contractguard/policy/`, `policies/`
08	LLM explanations	`contractguard/llm/`
09	Pipeline, report and CLI	`contractguard/pipeline.py`, `report/`, `cli.py`
10	Backend API	`backend/`
11	Streamlit UI	`frontend/`
12	Training the classifier	`training/`
13	Gold set and evaluation	`evaluation/`
14	Tests and tooling	`tests/`, `pyproject.toml`

The pipeline in one picture

```
file ──► ingestion ──► cleaning ──► segmentation ──► classification ──► metadata + facts
 (PDF/DOCX)  Document     CleanedText    list[Clause]     Clause.categories    ContractMeta,
Fact

         ──► policy engine ──► explainer ──► report builder ──► Report (JSON)
               list[Finding]     Finding.explanation
```

Every arrow carries a Pydantic model from `contractguard/models.py`, never a loose dictionary. The policy engine decides every status and the policy file decides every severity. The LLM only writes

explanations after that, and its answers are checked before they are shown.

01 Foundations: models, configuration, errors

Files: `contractguard/models.py`, `contractguard/config.py`, `contractguard/errors.py`, `config/*.yaml`, `.env.example`.

What

Three small modules that every other part of the system imports:

- **`models.py`** defines the Pydantic data models that pass between pipeline stages: `Block`, `Document`, `TextSpan`, `CleanedText`, `Clause`, `Party`, `ContractMeta`, `Fact`, `Explanation`, `Finding` and `Report`, plus the fixed vocabularies `CATEGORIES` (10 clause categories) and `CONTRACT_TYPES` (4 types).
- **`config.py`** loads the runtime settings and the YAML rule files.
- **`errors.py`** defines the exception classes, each with a stable error code.

Why

- **Typed contracts between stages.** If the segmenter returns a `list[Clause]`, the classifier can rely on every clause having `text`, `kind` and `char_start`. A typo in a field name fails at once instead of producing a silently empty report. The same models are reused by FastAPI for JSON, so the API documentation is generated from them.
- **No magic numbers in code.** Thresholds, regular expressions, label mappings and wording live in `config/` so they can be tuned and reviewed without touching Python (HLD 13).
- **Stable error codes.** The UI and API clients react to `error_code` (for example `ENCRYPTED_PDF`), while the human-readable `message` can change freely.

How

Models

Model	Produced by	Key fields
`Block`	ingestion	`text`, `page` (PDF) or `para_index` (DOCX), `is_ocr`, `ocr_confidence`, `style`
`Document`	ingestion	`file_type`, `num_pages`, `blocks`, `ocr_pages`, `warnings`
`CleanedText`	cleaning	`text`, `spans` (offset map back to blocks and pages), `removed_lines`
`Clause`	segmentation, then classification	`clause_id`, `number`, `heading`, `text`, `kind`, `page_start`, `section_path`, `char_start`, `char_end`, `categories`, `scores`, `low_confidence`, `classified_by`
`Party`, `ContractMeta`	metadata extraction	parties with alias and role, dates, term, governing law, title, contract type and its source
`Fact`	fact extraction	`key`, `value`, `unit`, `clause_ids`, `evidence_text`, `ambiguous`, `note`
`Finding`	policy engine	`rule_id`, `status`, `severity`, `expected`, `found`, `location`, `evidence_text`, `reason`, `explanation`
`Explanation`	explainer	five text fields, `quoted_evidence`, `source` (`llm` or `template`), `model`, `prompt_version`

Model	Produced by	Key fields
`Report`	report builder	everything above plus `counts`, `clause_checklist`, `warnings`, versions, `timings_s`, `disclaimer`

`CleanedText.span_at(offset)` and `pages_for(start, end)` turn a character position back into a page number. That is how a clause found in the cleaned text knows which PDF page it came from.

Configuration

`Settings` is a `pydantic-settings` class. Sources, highest priority first:

1. arguments passed to the constructor (used in tests);
2. environment variables with the `CG_` prefix and `__` for nesting, for example `CG_LLM__RPM=4` or `CG_OCR__TESSERACT_CMD=...`;
3. the short names from `.env.example`: `LLM_PROVIDER`, `LLM_MODEL`, `GEMINI_API_KEY`;
4. the `.env` file;
5. `config/settings.yaml`.

The API key is a `SecretStr`, so printing the settings shows asterisks. `get_settings()`, `get_patterns()`, `get_label_map()`, `get_contract_types()` and `get_templates()` are wrapped in `functools.lru_cache`, so each file is read once; `clear_caches()` resets them for tests. `Settings.resolve_path()` makes relative paths relative to the repository root, so the program works from any current directory.

File	Holds
`config/settings.yaml`	limits, OCR, segmentation lengths, model paths, thresholds, LLM pacing, organisation name, date order, disclaimer
`config/patterns.yaml`	every regular expression: cleaning, headings, parties, dates, durations, facts, SLA keywords
`config/label_map.yaml`	LEDGAR and CUAD label names mapped to ContractGuard categories
`config/contract_types.yaml`	keywords and weights for contract type detection
`config/templates.yaml`	reason sentences, labels and template explanations

Errors

`ContractGuardError(message, code)` is the base class. Subclasses:

- `IngestionError` with codes `UNSUPPORTED_FILE_TYPE`, `FILE_TOO_LARGE`, `TOO_MANY_PAGES`, `TOO_MANY_WORDS`, `ENCRYPTED_PDF`, `CORRUPT_FILE`, `OCR_UNAVAILABLE`, `EMPTY_DOCUMENT`;
- `PolicyError` with `POLICY_NOT_FOUND` and `POLICY_INVALID`;
- `ModelArtifactError` when the trained classifier files are missing.

The CLI prints `error [CODE]: message`; the backend stores the message on a failed job or returns it as `{error_code, message}`.

Callers and dependencies

Everything imports these modules; they import only Pydantic, pydantic-settings and PyYAML.

Errors and edge cases

- A missing YAML file raises `FileNotFoundError` at first use. A YAML file that is not a mapping raises `ValueError` with the path.

- A wrong setting type (for example `CG_LLM__RPM=abc`) fails validation when settings load, with a message naming the field.

Alternatives considered

- **Plain dictionaries between stages**: shorter to write, but errors surface far from their cause. Rejected.
- **dataclasses**: no validation or JSON support. Pydantic gives both, and FastAPI uses it.
- **Settings in Python constants**: rejected by the HLD's rule that thresholds are configuration.

Interview notes

- *Why Pydantic v2?* Validation from type hints, fast (its core is written in Rust), JSON serialisation for the API, and FastAPI uses the same models.
- *How are secrets handled?* Only in `.env` (git-ignored), loaded as `SecretStr`; the `config` command prints `gemini_api_key_set: true/false`, never the key.
- *Why `lru_cache` on loaders?* Parsing YAML and compiling regexes once per process keeps every stage fast; tests call `clear_caches()` so one test's environment does not leak into another.
- *What is `section_path`?* A location that works without pages, for example `7 > 7.2`. DOCX files have no pages, so findings in DOCX contracts are located by section.

02 Ingestion and OCR

Files: `contractguard/ingestion/router.py`, `pdf_loader.py`, `ocr.py`, `docx_loader.py`.

What

Turns an uploaded file into a `Document`: an ordered list of text `Block`s, each knowing its page (PDF) or paragraph index (DOCX), whether it came from OCR, and its OCR confidence.

Why

Contracts arrive as digital PDFs, scanned PDFs and Word files. Everything after this stage should work on plain text with a map back to where it came from, whatever the format. Scans are common in Indian procurement, so OCR is a first-class path, not an afterthought.

How

`router.py`: validate and dispatch

1. `detect_file_type(path)`: the extension must be `.pdf` or `.docx`, **and** the first bytes must match: `%PDF` for PDF, `PK\x03\x04` (a ZIP archive) for DOCX. A renamed `.exe` is rejected.
2. `check_size`: at most `limits.max_file_mb` (20 MB).
3. Load with the PDF or DOCX loader.
4. If no block has any text, raise `EMPTY_DOCUMENT`.

`pdf_loader.py`: text first, OCR only where needed

For each page:

1. Read text blocks with PyMuPDF (`page.get_text("blocks")`), keep text blocks only, and sort them top to bottom, then left to right.
2. `is_scanned`: the page is scanned if it has fewer than `ocr.min_chars_per_page` (50) characters **and** contains an image. Only those pages go to OCR, so a PDF that mixes typed pages and a scanned signature page works page by page.
3. Scanned page: `ocr_fn(page)` returns paragraphs and a mean confidence. Below `ocr.warn_confidence` (70), the document gets a warning such as "Page 3 OCR confidence 62% - verify manually".

Limits: encrypted PDFs (`ENCRYPTED_PDF`) and more than `limits.max_pages` (60) pages (`TOO_MANY_PAGES`) are rejected; a file PyMuPDF cannot open is `CORRUPT_FILE`.

`ocr.py`: Tesseract

1. `render_page`: render the page with PyMuPDF at `ocr.dpi` (300) in grayscale. No poppler needed.
2. `ocr_image`: `pytesseract.image_to_data` with `--oem 1 --psm 6` (LSTM engine, one uniform block of text). Words are grouped back into lines and paragraphs using Tesseract's block, paragraph and line numbers, so the paragraph structure survives.
3. Confidence = mean of word confidences above 0.
4. `tesseract_available()` is cached and always resets pytesseract's command first, so a custom path from one call never leaks into the next (a bug found by a test).

`docx_loader.py`: rebuild Word's automatic numbering

Word stores "7.2" outside the paragraph text, in the numbering definitions. Without them, the segmenter would see a heading with no number.

1. `Numbering.from_document` reads `numbering.xml`: for each list, each level's start value, format (decimal, letters, roman) and label template such as `%1.%2`.
2. For each numbered paragraph (`w:numPr` on the paragraph or inherited from its style), `Numbering.label` advances that level's counter, resets deeper levels, and renders the label.
3. Paragraphs and tables are read in document order. A table row becomes one block with cells joined by `|` (merged cells are de-duplicated).
4. Each block keeps its Word style name (for example "Heading 2") as a hint for segmentation.
5. More than `limits.max_docx_words` (60,000) words raises `TOO_MANY_WORDS`.

DOCX has no pages, so `page` stays `None`; findings in DOCX contracts are located by section.

Callers and dependencies

Called by `pipeline.run` through `load_document`. Uses PyMuPDF, pytesseract with the Tesseract 5 binary, Pillow and python-docx.

Data in and out

In: a file path and `Settings`. Out: `Document(file_type, num_pages, blocks, ocr_pages, warnings)`.

Errors and edge cases

Situation	Result
Wrong extension or content does not match it	`UNSUPPORTED_FILE_TYPE`
Password-protected PDF	`ENCRYPTED_PDF`
Scanned page but Tesseract missing	`OCR_UNAVAILABLE` with install instructions
Low OCR confidence	warning in the report, analysis continues
Two-column layouts	blocks are sorted by position; columns can interleave (known limit)

Configuration

`limits.*` and `ocr.*` in `settings.yaml`; `CG_OCR__TESSERACT_CMD` for a custom Tesseract path (common on Windows).

Alternatives considered

- **pdfplumber or pdfminer**: slower, and no page rendering for OCR. PyMuPDF does both.
- **OCR every page**: slower and less accurate than reading the embedded text of digital pages.
- **Cloud OCR (Google Vision, AWS Textract)**: more accurate, but sends whole contracts off the machine and costs money. Tesseract runs locally for free.
- **docx2txt**: loses automatic numbering, which segmentation depends on.

Interview notes

- *How do you detect a scanned page?* Less than 50 characters of extractable text plus an image on the page. Checked per page, not per document.

- *What do `--oem 1 --psm 6` mean?* OCR engine mode 1 is the LSTM neural engine; page segmentation mode 6 treats the page as one uniform block of text, which suits contract pages.
- *Why render at 300 DPI?* Tesseract is most accurate around 300 DPI; lower loses small fonts, higher costs time without gains.
- *Measured OCR quality?* Character error rate 1.10% on the simulated scans of the 8 gold contracts. Simulated scans are cleaner than real ones, so this is optimistic (D16).
- *Why check magic bytes?* The extension is user-controlled; the first bytes are what parsers act on. It stops renamed files reaching the wrong parser.

03 Cleaning and normalisation

Files: `contractguard/preprocessing/cleaner.py`, `normalizer.py`; patterns in `config/patterns.yaml: cleaning`.

What

Turns a `Document` into `CleanedText`: one string with blocks separated by a blank line, plus a list of `TextSpan`s that map every stretch of the string back to its block, page and paragraph.

Why

Raw PDF text is full of noise that breaks later stages: running headers ("Master Services Agreement - Confidential") repeated on every page, "Page 3 of 9" lines, words hyphenated across line breaks, sentences broken into short lines, curly quotes and odd dashes. Regular expressions for facts and headings are far simpler on clean text. The offset map keeps page numbers available for the report even though the text has changed.

How

`clean_document(doc)` runs these steps:

1. **Repeating headers and footers** (PDF with at least 3 pages): `find_repeating_lines` looks at the top 3 and bottom 3 non-empty lines of each page. A line's key has its digits removed and its spaces collapsed, so "Page 2 of 9" and "Page 3 of 9" match. A key that appears in the same zone on at least half of the pages is a header or footer, and only lines inside those zones are dropped. **Top and bottom are counted separately (D23)**: a sentence that wraps across a page break ("... fifteen" / "(15) days' notice" at the foot of one page) once looked like a repeated line and was deleted. That bug removed real notice periods from GC04 and was found by the fact evaluation.
2. **Page-number lines** such as "3", "Page 3" or "3 of 9" are removed.
3. **De-hyphenation**: "termi-\nnation" becomes "termination".
4. **Soft-wrap joining** inside a block: a line that does not end a sentence, followed by a line that starts in lower case (or with "(15) days"), is joined with a space (D24). A clause number left alone on its line ("7.2") is joined to the next line.
5. **Character normalisation** (`normalize_chars`): curly quotes to straight quotes, all dashes to "-", non-breaking and other spaces to a plain space, soft hyphens removed, repeated spaces collapsed, then Unicode NFKC.

Steps 2 to 5 run per block, so each kept block becomes exactly one `TextSpan`.

Callers and dependencies

Called by `pipeline.run`. Consumed by the segmenter, which uses `CleanedText.span_at` and `pages_for` to give clauses page numbers.

Data in and out

In: `Document`. Out: `CleanedText(text, spans, file_type, removed_lines)`. `removed_lines` lists the dropped headers for debugging.

Errors and edge cases

- Short documents (1 to 2 pages) skip header removal: with so few pages, repetition is not evidence (D15). A header on a 2-page contract stays in the text; the segmenter tolerates it.
- Case, stop words and punctuation are kept on purpose: "not", "unless" and "except" change legal meaning. Lowercasing happens only inside the TF-IDF vectoriser.

Configuration

`cleaning.header_footer_lines` (3), `header_footer_min_ratio` (0.5), `header_footer_min_pages` (3), and the regexes `page_number_line`, `dehyphenate`, `soft_wrap_end`, `soft_wrap_next`, `number_only_line`.

Alternatives considered

- **Remove the first and last line of every page blindly**: deletes real text on pages without a header.
- **Layout analysis with font sizes and positions**: more accurate but much more complex, and OCR output has no font information.
- **Lowercasing and stop-word removal here**: would destroy negations needed by fact extraction.

Interview notes

- *Why keep an offset map?* Cleaning changes character positions; the map lets any clause found later report the page it came from.
- *What is NFKC?* Unicode compatibility normalisation: it folds look-alike characters (for example full-width digits or the "fi" ligature) into their plain forms, so regexes match.
- *Tell me about a bug here.* D23: a sentence wrapped across a page break was taken for a running header and deleted, which removed notice periods. Counting the top and bottom zones separately fixed it, and a regression test reproduces the exact page layout.

04 Clause segmentation

Files: `contractguard/segmentation/segmenter.py`; patterns in `config/patterns.yaml: segmentation`.

What

Splits the cleaned text into `Clause`s in reading order, with ids `c001`, `c002`, ..., the clause number ("7", "7.2", "VII"), a heading, the text, the page range, a `section_path` and a kind: `PREAMBLE`, `BODY`, `DEFINITIONS`, `EXHIBIT` or `SIGNATURE`.

Why

Every later decision is made per clause: the classifier labels clauses, facts are read from the clauses of one category, and findings point at clauses. If two clauses are merged, a termination clause can hide inside a payment clause; if one clause is split badly, its sentence loses its subject.

How

1. **Choose a mode.** Count numbered headings (level 1 or 2 patterns, not level 3). With fewer than `segmentation.fallback_min_headings` (3), use the **paragraph fallback**: split on blank lines and merge paragraphs shorter than `fallback_merge_chars`.
2. **Scan lines** (`_scan`): - A level 1 heading ("7. Payment", "ARTICLE VII", "SECTION 7", or "7 TERM AND RENEWAL") or level 2 heading ("7.2") starts a clause. Level 3 items ("(a)", "(i)", "7.2.1") are appended to their parent. - For DOCX, a paragraph whose Word style matches `Heading*` also starts a clause. - An exhibit heading ("Exhibit A", "Schedule 1", "Annex", "Appendix") starts an `EXHIBIT` clause; clauses after it stay exhibits. - "IN WITNESS WHEREOF" (or a line starting "Signature" or "By:") starts the `SIGNATURE` block. - Text before the first clause is the `PREAMBLE` (parties and date live here). - A heading containing "Definitions" or "Interpretation" makes the clause `DEFINITIONS`.
3. **Parse the start line** (`parse_start`): split the number from the rest. The rest is a heading only if it is all capitals, or short Title Case text not ending like a sentence (D18). "Contract Price and Payment" is a heading; "On termination, the Contractor shall withdraw its" is a wrapped sentence. ALL-CAPS headings are converted to Title Case.
4. **OCR-tolerant Roman numerals** (D17): Tesseract reads the "I" of "ARTICLE III" as `l`, `1`, `i` or `|`. `_roman_fix` maps those back, so "ARTICLE Ill" becomes "III".
5. **Length guards.** Clauses under `min_chars` (40) merge into the next clause. Clauses over `max_chars` (3,000) are split at sentence boundaries into parts with suffixed ids (`c014a`, `c014b`); a single sentence longer than the limit stays whole.
6. **Locate.** Each clause gets `page_start` and `page_end` from the offset map, `is_ocr` if any part came from OCR, and a `section_path` such as `7 > 7.2`.

Callers and dependencies

Called by `pipeline.run`. Uses the spaCy sentencizer from `extraction/sentences.py` for long-clause splits.

Data in and out

In: `CleanedText` and, optionally, the `Document` (for DOCX styles). Out: `list[Clause]` with classification fields still empty.

Errors and edge cases

- Contracts without numbering fall back to paragraphs, so they are still segmented, just less precisely.
- A running header that survived cleaning (2-page contracts) is absorbed into a clause; evaluation matching tolerates it.
- Lettered sub-items are deliberately merged into their parent, matching how reviewers cite clauses ("clause 7.2"), at the cost of very long clauses for heavily nested contracts.

Configuration

`segmentation.min_chars`, `max_chars`, `fallback_min_headings` in `settings.yaml`; `segmentation.l1`, `l2`, `l3`, `parse`, `definitions_heading`, `exhibit_heading`, `signature`, `docx_heading_style`, `heading_max_words`, `fallback_merge_chars` in `patterns.yaml`.

Alternatives considered

- **A trained sequence model (CRF or transformer) for boundaries**: needs labelled boundary data, which does not exist for Indian contracts. Numbered headings are reliable in real contracts.
- **Fixed-size chunks**: break clauses mid-sentence and lose the clause number reviewers need.
- **One clause per paragraph**: splits multi-paragraph clauses.

Interview notes

- *Measured result?* Boundary F1 1.000 on DOCX and 0.986 on text and scanned PDFs of the 8 gold contracts. Optimistic, because the patterns were developed on these contracts (D21).
- *How are boundaries scored?* A predicted clause matches a gold clause when its opening contains the gold clause's first eight words with a rapidfuzz partial ratio of at least 85, one to one and in order.
- *Hardest case?* OCR turning Roman numerals into `l` and `|`, and wrapped sentences that look like headings; D17 and D18 handle them.

05 Clause classification and embeddings

Files: `contractguard/classification/classifier.py`, `features.py`, `contractguard/embeddings/service.py`; artifacts in `models/`.

What

Gives each `BODY` and `EXHIBIT` clause zero or more categories from `CONFIDENTIALITY`, `TERMINATION`, `PAYMENT`, `LIABILITY`, `INDEMNIFICATION`, `IP`, `GOVERNING_LAW`, `RENEWAL`, `SLA` and `OTHER`, with a score per category and a note of how it was decided (`classified_by`).

Why

The policy engine asks "is there a liability clause, and what does it say?". Fact extractors only read clauses of their own category, which keeps them precise: a "30 days" in a payment clause must not be read as a notice period. Clauses can belong to several categories (a termination clause that also covers confidentiality after termination), so this is **multi-label** classification.

How

For each classifiable clause, `ClauseClassifier.classify`:

1. **Trained classifier.** `predict_proba` from the saved scikit-learn pipeline (TF-IDF features and a calibrated linear SVM per category). A category is assigned when its probability reaches that category's tuned threshold (`models/thresholds.json`, between 0.3 for RENEWAL and 0.8 for INDEMNIFICATION).
2. **SLA keyword rule.** Neither training dataset labels service levels, so a clause matching `patterns.yaml: sla_keywords` ("service level", "SLA", uptime wording) gets `SLA`. If the classifier found nothing else, `classified_by` becomes `rule`.
3. **Embedding fallback.** A clause still without a category is embedded with MiniLM and compared with one prototype vector per category (the mean embedding of up to 200 training examples). If the best cosine similarity is at least `embeddings.proto_threshold` (0.80, D50), that category is assigned with `low_confidence=True` and `classified_by="embedding_fallback"`. The policy engine turns findings based only on low-confidence clauses into NEEDS_REVIEW.
4. Otherwise the clause is `OTHER`.

`PREAMBLE`, `DEFINITIONS` and `SIGNATURE` clauses are not classified.

`features.py`

Code that must be importable when the pickled model loads:

- `legal_stop_words()`: scikit-learn's English stop words **minus** negations ("no", "not", "nor", "never", "none", "without", "unless", "except", "notwithstanding").
- `MaskedOneVsRest` (D39): one binary classifier per category, trained only on rows where that category's label is known. LEDGAR has no LIABILITY or RENEWAL label and CUAD has no CONFIDENTIALITY, PAYMENT or INDEMNIFICATION label; a CUAD paragraph about confidentiality is therefore *unknown* (-1) for CONFIDENTIALITY, not a negative example. Without masks the model learned that TERMINATION never happens in CUAD-style text.
- `MiniLMEncoder`: a scikit-learn transformer wrapping the embedding model, used by the MiniLM comparison model.

`embeddings/service.py`

- `EmbeddingService`: lazy, thread-safe wrapper around `SentenceTransformer` (`sentence-transformers/all-MiniLM-L6-v2`, 384 dimensions) on CPU. It tries the local cache first, so no network call happens once the model is downloaded. Vectors are L2-normalised, so cosine similarity is a dot product.
- `cosine_top_k(query, matrix, k)`: a NumPy matrix product and a stable sort; no vector database.
- `try_load_embedder`: returns `None` instead of failing; the pipeline then adds a warning and skips the semantic steps.

Callers and dependencies

`pipeline.Resources.load` calls `load_classifier(settings, embedder)`; `pipeline.run` calls `classify`. The policy engine reuses the embedder for its semantic search. Depends on scikit-learn, joblib, NumPy and sentence-transformers (PyTorch).

Data in and out

In: `list[Clause]` from segmentation. Out: the same clauses with `categories`, `scores`, `low_confidence` and `classified_by` filled.

Errors and edge cases

- Missing `classifier.joblib` or `thresholds.json`: `ModelArtifactError`, and the run stops. Without a classifier every required clause would be reported MISSING, a confidently wrong report (D32).
- Missing prototypes or embedding model: the fallback is skipped, with a warning.
- The pickled model must be loaded with the same scikit-learn version that saved it (pinned in `deploy/requirements.txt`).

Configuration

`classification.model_path`, `thresholds_path`, `embeddings.model_name`, `proto_threshold`, `prototypes_path` in `settings.yaml`; `sla_keywords` and `sla_prototypes` in `patterns.yaml`.

Alternatives considered

Five models were compared on the validation split (macro-F1 at threshold 0.5):

Model	Best validation macro-F1
TF-IDF + calibrated linear SVM (C = 1)	**0.857** (selected; 0.875 with tuned thresholds)
TF-IDF + logistic regression (C = 64)	0.846
TF-IDF + random forest	0.815
TF-IDF + multinomial naive Bayes	0.716
MiniLM embeddings + logistic regression	0.607

Fine-tuning a transformer (Legal-BERT) was out of scope: no GPU, slower inference, and the linear model already beats the 0.75 target.

Interview notes

- *Result on the test split (E1)?* Macro-F1 0.854, micro-F1 0.869 over 10,399 test rows. Weakest categories: TERMINATION (F1 0.73) and PAYMENT (0.76); most errors are missed labels

(predicted "none"), not confusions between categories.

- *Why does the SVM need calibration?* A linear SVM outputs distances, not probabilities. `CalibratedClassifierCV` with sigmoid (Platt) scaling and 3-fold cross-validation turns them into probabilities, so per-category thresholds mean something.
- *Why per-category thresholds?* Categories differ in frequency and difficulty. Each threshold is the value in 0.2 to 0.8 (step 0.05) with the best validation F1 for that category.
- *Why did MiniLM lose?* Its general-purpose sentence embeddings blur legal distinctions that specific words and bigrams capture ("limitation of liability", "governed by"), and a linear model on 384 dense features has less to work with than on 100,000 sparse ones.
- *Why keep negations out of the stop-word list?* "shall not be liable" and "shall be liable" must not look the same.

06 Fact and metadata extraction

Files: `contractguard/extraction/sentences.py`, `durations.py`, `dates.py`, `parties.py`, `entities.py`, `facts.py`, `document.py`, and `contractguard/classification/contract_type.py`; patterns in `config/patterns.yaml`.

What

Reads structured values out of the classified clauses:

- **Contract metadata** (`ContractMeta`): title, contract type, the parties with their aliases and roles (our organisation or counterparty), effective date, expiry date, term text and governing law.
- **Seven facts** for the policy rules: `termination.notice_days`, `termination.counterparty_unilateral`, `liability.has_cap`, `liability.is_uncapped`, `renewal.is_automatic`, `payment.due_days`, `governing_law.jurisdiction`.

Every fact carries the clause ids it came from and the **whole sentence** that matched, copied verbatim from the clause, as evidence.

Why

Policy rules are about values ("notice of at least 30 days"), not about clause presence alone. Rules and regular expressions are used instead of a model because there is no labelled data for these values, the patterns in contracts are formulaic, and every result must be explainable by pointing at the exact sentence.

How

Building blocks

- **`sentences.py`**: spaCy's rule-based sentencizer on each line. Splits after abbreviations ("Pvt.", "Ltd.", "No.") or before a lower-case start are merged back. Returns character spans, so every sentence is an exact substring of the clause.
- **`durations.py`**: finds durations such as "thirty (30) days", "30 business days", "two months" and "Net 45". Digits win over words when both are present. Units convert to days (week 7, month 30, year 365); business days become calendar days as ceil(n × 7 / 5), so 30 business days is 42 days.
- **`dates.py`**: `dateparser` with day-first order by default (`org.date_order: DMY`, Indian convention). Finds the effective date, an explicit expiry date, or a term ("for a period of two (2) years from the Effective Date") and computes the end with the **legal reading** (D9): the day before the anniversary, so 1 April 2026 for one year ends on 31 March 2027; a 29 February start ends on the last day of February.
- **`parties.py`**: a preamble regex finds "between X ... (the "Alias") and Y ... (the "Alias")". Each name is cut where its description starts (", a company incorporated ...", "having its office ..."); aliases survive OCR losing the opening quote mark. Roles: a party whose name fuzzy-matches our organisation's name (rapidfuzz `token_set_ratio` at least 85) is `OUR_ORG`, the other is `COUNTERPARTY`; otherwise aliases decide ("Company", "Client", "Customer", "Licensee", "Buyer" are ours; "Vendor", "Supplier", "Provider", "Contractor", "Licensor", "Seller" are theirs).
- **`entities.py`**: when the preamble regex fails, spaCy `en_core_web_sm` ORG entities supply the first two organisations as parties.
- **`contract_type.py`**: keyword scores; a hit in the first 3 lines counts 3, a hit in the first 1,500 characters counts 1; under 2 points means `OTHER`. A type chosen by the user always wins.

The facts (`facts.py`)

Fact	Read from	Method
`termination.notice_days`	TERMINATION clauses	Durations within 12 tokens of "notice" / "prior written" in sentences about terminating. A duration right after "within" is a cure period and is skipped. If some notices are for convenience ("for convenience", "without cause", "at any time"), only those count. Several values: the **minimum**, because the weakest protection is the risk.
`termination.counterparty_unilateral`	TERMINATION	True only when every convenience-termination right belongs to the counterparty (its name or alias is the subject); "either party" makes it False.
`liability.has_cap`, `liability.is_uncapped`	LIABILITY	Cap phrases ("shall not exceed", "limited to", "maximum aggregate liability") and uncapped phrases ("unlimited liability", "liable for all losses"). A negation cue within 8 tokens before a match cancels it; "including but not limited to" never counts as a cap.
`renewal.is_automatic`	RENEWAL	Automatic wording ("shall automatically renew", "evergreen", "unless notice of non-renewal") versus manual ("may be renewed by mutual agreement"). Both found: ambiguous.
`payment.due_days`	PAYMENT	In sentences about invoices or payment, the duration right after "within", "net" or "no later than". Different values: ambiguous.
`governing_law.jurisdiction`	GOVERNING_LAW	"laws of X", normalised with aliases: "Karnataka" becomes "India (Karnataka)", "England and Wales" becomes "UK (England and Wales)". Different values: ambiguous.

A fact with no clause of its category has value `None` and no evidence; the policy engine decides whether that means MISSING or NOT_APPLICABLE. **Ambiguous facts get value `None` and `ambiguous=True`**, never a guess, and the engine turns them into NEEDS_REVIEW.

`document.py`

`extract_meta(clauses, our_org, contract_type, date_order)` joins the pieces: the preamble (or the first characters of the text) for parties and dates, contract type detection on the full text, the title from the first line, and governing law from the fact extractor.

Callers and dependencies

`pipeline.run` calls `extract_meta` and then `extract_facts(clauses, meta.parties)`. Uses spaCy, dateparser and rapidfuzz.

Data in and out

In: classified `list[Clause]`, our organisation's name, optional user-chosen type, date order. Out: `ContractMeta` and `dict[str, Fact]` keyed by fact name.

Errors and edge cases

- Party roles unresolved (our name not found, unusual aliases): a report warning, and role-dependent rule R04 may need review.
- Unknown jurisdictions are kept as written and fail the "India only" rule, which is the safe direction.
- Known limits: double negation, notice periods stated in a schedule far from the termination clause, and amounts of money (no V1 rule uses them, D27).

Configuration

`patterns.yaml` sections `parties`, `dates`, `durations`, `governing_law`, `facts`, `sentences`; `contract_types.yaml`; `org.default_name` and `org.date_order` in `settings.yaml`.

Alternatives considered

- **LLM extraction**: flexible, but values could be invented and results would change between runs. The design rule is that the LLM never decides status, so it cannot supply the values the status depends on.
- **A trained NER model for durations and money**: no labelled data, and durations are matched more precisely by `durations.py` (D27).
- **Effective date plus 365 days for expiry**: off by one day and wrong in leap years (D9).

Interview notes

- *Measured accuracy (E4)?* 100% of the 7 facts and the metadata on all 3 formats of the 8 gold contracts, using gold clause categories. Optimistic: the patterns were written while looking at these contracts (D29). End to end, the policy result (E5) is the better measure.
- *How is negation handled?* A window rule similar to NegEx: a cue such as "not", "no", "in no event" or "neither" within 8 tokens before the match, in the same sentence, cancels it.
- *Why the minimum notice period?* With 15 days for the vendor and 30 for us, our exposure is the 15-day exit, so the conservative reading is the minimum, and it is not marked ambiguous.
- *Why whole sentences as evidence?* A reviewer needs the sentence, and because it is a verbatim substring, the LLM's quote can be checked against it (D25).

07 Policy engine

Files: `contractguard/policy/engine.py`, `schema.py`; `policies/default_vendor_v1.json`, `policies/policy.schema.json`, `policies/history/`; wording in `config/templates.yaml`.

What

Checks a contract against an organisation's policy and returns one `Finding` per rule with a status (`NON_COMPLIANT`, `MISSING`, `NEEDS_REVIEW`, `COMPLIANT`, `NOT_APPLICABLE`), a severity, what was expected, what was found, where, the evidence and a plain-English reason.

Why

This is the part that must be trustworthy. It is **deterministic**: the same contract and policy always give the same findings, every decision can be traced to a clause and a fact, and the rules are a JSON file a legal team can read and edit. **The LLM is never consulted here** (HLD 0.3).

How

The policy file

`default_vendor_v1.json` (version 1.0.0) holds 10 rules:

Rule	Category	Check	Expectation	Severity	Applies to
R01	Liability	BOOLEAN_EQUALS `liability.has_cap`	true	HIGH	Vendor, SaaS
R02	Liability	BOOLEAN_EQUALS `liability.is_uncapped`	false	HIGH	Vendor, SaaS
R03	Termination	NUMERIC_MIN `termination.notice_days`	at least 30	HIGH	Vendor, SaaS
R04	Termination	BOOLEAN_EQUALS `termination.counterparty_unilateral`	false	MEDIUM	Vendor, SaaS
R05	Renewal	BOOLEAN_EQUALS `renewal.is_automatic`	false	MEDIUM	all types
R06	SLA	CLAUSE_PRESENT	present	MEDIUM	Vendor, SaaS
R07	Confidentiality	CLAUSE_PRESENT	present	HIGH	all types
R08	Governing law	ENUM_IN `governing_law.jurisdiction`	India	MEDIUM	all types
R09	Payment	NUMERIC_MIN `payment.due_days`	at least 30	LOW	Vendor, SaaS
R10	Indemnification	CLAUSE_PRESENT	present	MEDIUM	Vendor, SaaS

Each rule also has `policy_text` (the policy in words, also used as the semantic search query) and `recommendation_template` (the review action shown to the reviewer).

Evaluating one rule (`PolicyEngine.evaluate_rule`)

1. **Contract type.** Not in `applies_to`: `NOT_APPLICABLE`.

2. **Matching clauses.** `BODY` or `EXHIBIT` clauses whose categories include the rule's category.
3. **None found.** Semantic search: embed `policy_text`, compare with every body clause, and if the best cosine similarity reaches `policy.tau_missing` (0.55), return `NEEDS_REVIEW` pointing at that clause ("possibly covered by clause 9"). Otherwise `MISSING` if the rule needs the clause (CLAUSE_PRESENT or `required_clause`), else `NOT_APPLICABLE`.
4. **Only low-confidence clauses** (from the embedding fallback): `NEEDS_REVIEW`.
5. **CLAUSE_PRESENT**: `COMPLIANT`.
6. **The fact.** Missing, ambiguous or `None`: `NEEDS_REVIEW` ("could not be determined").
7. **Compare** (`compare`): BOOLEAN_EQUALS is an identity test; NUMERIC_MIN is `value >= threshold`, NUMERIC_MAX is `value <= threshold`; ENUM_IN checks whether any allowed value appears in the value ("India (Karnataka)" contains "India"). Pass: `COMPLIANT`; fail: `NON_COMPLIANT`. A value of the wrong type for the check gives `NEEDS_REVIEW`.

Severity comes from the rule for NON_COMPLIANT and MISSING, is LOW for NEEDS_REVIEW, and is empty otherwise. **Location** is "Page N" / "Pages N-M" for PDFs and "Section X" for DOCX. **Reasons** come from `templates.yaml`, for example "Termination notice is 7 days; the policy requires at least 30 days." Findings are sorted by status, then severity, then clause position.

Policy files (`schema.py`)

- `validate_policy` checks a policy against `policy.schema.json` (JSON Schema draft 2020-12) and reports every error in one message, plus duplicate rule ids.
- `load_policy` only accepts ids made of letters, digits and underscores (no path tricks).
- `save_policy` (used by `PUT /policies/{id}`) validates the new version, copies the current file to `policies/history/{id}@{version}.json`, and bumps the patch version. Every report records the policy version it used.

Callers and dependencies

`pipeline.run` creates `PolicyEngine(policy, embedder, settings)` and calls `evaluate`. The backend policy endpoints call `list_policies`, `load_policy` and `save_policy`. Uses `jsonschema` and the embedding service.

Data in and out

In: contract type, classified clauses, facts. Out: `list[Finding]` without explanations.

Errors and edge cases

- No embedding model: step 3's search is skipped, so an empty category goes straight to MISSING or NOT_APPLICABLE, and the report carries a warning.
- A policy file that fails the schema: `POLICY_INVALID`, and the edit is not saved.

Configuration

`policy.default_policy_id`, `policy.tau_missing` (a policy file can override it in `thresholds.tau_missing`), `policy.policies_dir`.

Alternatives considered

- **Ask the LLM whether a clause complies**: not reproducible, not auditable, and can be wrong with confidence. Rejected by design.

- **A rules engine library (Drools, durable-rules)**: more power than 5 check types need, and a harder file format for legal teams.
- **Hard-coded rules in Python**: a policy change would need a code change and a release.

Interview notes

- *Measured result (E5)?* End to end with the trained classifier: violation recall 88.9% (16 of 18 violations found) and precision 100% on all three formats; exact status accuracy 93.8%. The misses are GC03's missing SLA and indemnity clauses, reported as NEEDS_REVIEW (semantic search found a similar clause) instead of MISSING: cautious, not silent.
- *What counts as a violation?* NON_COMPLIANT or MISSING.
- *Why NEEDS_REVIEW instead of a guess?* A wrong COMPLIANT hides risk and a wrong NON_COMPLIANT wastes time and trust. "Please look" is the honest answer when the evidence is unclear.
- *How do you prevent a policy edit from breaking the system?* Schema validation before saving, versioned history, and the version recorded in every report.

08 LLM explanations

Files: `contractguard/llm/base.py`, `gemini.py`, `ollama.py`, `mock.py`, `prompts.py`, `explainer.py`; wording for templates in `config/templates.yaml`.

What

Adds a plain-English `Explanation` to every flagged finding (NON_COMPLIANT, MISSING, NEEDS_REVIEW) and writes a short executive summary. Each explanation has a one-sentence summary, 2 to 4 sentences of explanation, why it matters, a suggested review action, and a quote from the clause.

Why

A finding such as "R03 NON_COMPLIANT, found 7 days" is precise but terse. Reviewers, especially non-lawyers in procurement, understand faster with a short explanation tied to the clause. LLMs write that well. They can also invent things, so the design gives them **no power over results** and **checks every answer** before showing it.

How

Providers (`base.py`, `gemini.py`, `ollama.py`, `mock.py`)

All implement one interface, `generate_json(system, user, schema, temperature, timeout_s)`, and raise `LLMError(retryable=...)` on failure:

- **Gemini** through the `google-genai` SDK with JSON output constrained by a response schema. HTTP 408, 429 and 5xx are retryable.
- **Ollama** through its local HTTP API (`POST /api/chat` with the JSON schema as `format`), using only the standard library. Nothing leaves the machine.
- **Mock** builds a grounded answer from the prompt itself, or replays scripted answers and exceptions; used by tests and offline demos.

`make_provider(settings)` returns `None` (templates only) when the provider is `none`, the model name is missing, or the Gemini key is missing.

Prompts (`prompts.py`)

The system prompt says: do not give legal advice, do not change status or severity, use only the clause and policy given, return only JSON. The user prompt carries the contract type, the party aliases, the rule and policy text, the automated result (status, severity, expected, found, location) and the clause text. The version string `explain-v1` is part of the cache key, so any wording change must bump it.

The explainer (`explainer.py`)

For each flagged finding, the top `llm.max_findings` (15) by severity go to the LLM; the rest get template explanations.

1. **Cache.** Key = SHA-256 of prompt version, rule, status, clause text, expected, found and model. A hit skips the call. The cache is the `llm_cache` SQLite table.
2. **Pacing.** A token-bucket `RateLimiter` allows at most `llm.rpm` (8) calls per minute, to stay inside free-tier limits.

3. **Retries.** Retryable errors back off 2 s, 4 s, 8 s (`llm.max_retries` = 3).
4. **Grounding checks** on the answer: - `quoted_evidence` must be an exact substring of the clause after whitespace normalisation, and empty for MISSING findings (there is no clause to quote); - every number in the prose fields must appear somewhere in the prompt (D37), so an invented "45 days" cannot reach the reviewer.
5. **One retry** with a correction note if a check fails; after that, the **template explanation** is used. The template is built from the finding's own reason, the policy text and the rule's recommendation, so it is always correct, just less fluent.
6. **Executive summary.** One call listing the flagged findings; its numbers are checked too. On failure, a template summary counts the results and names the top three issues.

Status, severity, reason, expected and found are never changed by this stage.

Callers and dependencies

`pipeline.run` creates an `Explainer` after the policy engine. If the whole stage raises, the pipeline retries with templates only and adds a warning: a report is never lost over explanations.

Data in and out

In: findings, rules, clauses, metadata. Out: the same findings with `explanation` filled, and the summary text. `Report.llm_used` says whether any explanation came from the LLM.

Errors and edge cases

Situation	Result
No key, no model, provider `none`	templates, warning "AI explanations are off"
Rate limit or timeout	backoff and retry, then template for that finding
Answer quotes text not in the clause	one retry, then template
Answer contains a number not in the input	one retry, then template
LLM returns invalid JSON	counted as an error, then template

Configuration

`llm.provider` (`gemini`, `ollama`, `mock`, `none`), `llm.model`, `llm.ollama_host`, `llm.rpm`, `llm.max_findings`, `llm.temperature` (0.2), `llm.timeout_s`, `llm.max_retries`; environment variables `LLM_PROVIDER`, `LLM_MODEL`, `GEMINI_API_KEY`.

Alternatives considered

- **One call for the whole report**: sends much more of the contract, harder to ground, and fails all at once.
- **Free-text answers**: harder to check; a JSON schema makes each field testable.
- **Let the LLM decide compliance**: rejected; see the policy engine guide.
- **Retrieval-augmented chat over the contract**: out of scope for V1; the "retrieval" here is the exact evidence clause chosen by the deterministic pipeline.

Interview notes

- *What did you measure (E7)?* With a local Qwen 2.5 3B model through Ollama: 18 of 20 flagged findings got an LLM explanation that passed the checks (90%), 2 fell back to templates, and no invented numbers reached the report. An AI rating gave 4.0 out of 5 for faithfulness; human ratings are still to be collected, so that score is provisional.
- *How do you stop hallucinations?* Narrow input (one clause), JSON schema, an exact-substring check on the quote, a check that every number exists in the input, one retry, then a template.
- *Privacy?* With Gemini, only the single clause for a finding is sent. With Ollama, nothing leaves the machine. Free-tier Gemini data may be used by Google, so demos use synthetic contracts.
- *Why temperature 0.2?* Low randomness keeps answers close to the input and repeatable, while still reading naturally.

09 Pipeline, report and CLI

Files: `contractguard/pipeline.py`, `contractguard/report/builder.py`, `contractguard/cli.py`.

What

- **`pipeline.py`** runs all stages in order and returns a `Report`.
- **`report/builder.py`** assembles the final `Report`: checklist, counts, warnings, versions, timings and the disclaimer.
- **`cli.py`** is the command-line entry point: `contractguard analyze <file>` and `contractguard config`.

Why

One orchestrator means the CLI, the API, the tests and the evaluation scripts all run exactly the same code path. Loading slow resources once (the classifier, MiniLM, the LLM client) and reusing them is what makes the API fast after startup.

How

`Resources` and `Options`

- `Options`: per-run choices: contract type (or auto), our organisation's name, policy id, whether to use the LLM.
- `Resources.load(settings, with_embeddings, with_llm, cache_path)`: loads the embedder (or records a warning if it is unavailable), the classifier (raises if missing), the LLM provider and the LLM cache. The backend calls this once at startup; the CLI once per run.

`run(path, options, resources, progress_cb, contract_id, filename)`

Step	Progress	Work
EXTRACTING	5%	`load_document`, then `clean_document`
SEGMENTING	30%	`segment`
CLASSIFYING	45%	`classifier.classify`
EVALUATING	65%	load the policy; our organisation = option, else the policy's default, else the setting; `extract_meta`, `extract_facts`, `PolicyEngine.evaluate`
EXPLAINING	80%	`Explainer.explain_findings` and the executive summary
DONE	100%	`build_report`

Each stage is timed with `time.perf_counter()` and the timings go into `Report.timings_s` (`extract`, `segment`, `classify`, `evaluate`, `explain`).

Failure policy (HLD 16). Ingestion errors and a missing classifier stop the run with a readable `ContractGuardError`. A missing embedder, unresolved party roles, or a failing LLM only add warnings; the report is still produced.

Report builder

- `clause_checklist`: for each category except OTHER, whether at least one body or exhibit clause has it.

- `counts`: `HIGH`, `MEDIUM`, `LOW` count the issues (NON_COMPLIANT and MISSING) by severity; each status has its own count; `CLAUSES` counts body and exhibit clauses (D41).
- Warnings from ingestion and the pipeline are merged without duplicates.
- Records `policy_id`, `policy_version`, `pipeline_version`, `generated_at` (UTC), the disclaimer, file type, page count, OCR pages and whether the LLM was used.

CLI

```
contractguard analyze contract.pdf --org "ABC Technologies Pvt. Ltd." --no-llm -o report.json
contractguard config        # effective settings, API key shown only as set / not set
```

`analyze` prints the title, type, parties, counts, every flagged finding with its location and reason, the warnings and the disclaimer. Exit codes: 0 success, 1 a ContractGuard error (printed as `error [CODE]: message`), 2 file not found.

Callers and dependencies

Called by the CLI, `backend/jobs.py`, and the evaluation scripts. Calls every `contractguard` stage.

Data in and out

In: a file path, `Options`, `Resources`, an optional progress callback. Out: a `Report`.

Errors and edge cases

- Progress callbacks let the backend show the stage in the UI; the CLI passes none.
- If explanation fails as a whole, the explainer is rebuilt without a provider and templates are used, with a warning naming the exception type.

Configuration

Reads `Settings` through `Resources`; `--org`, `--policy`, `--contract-type`, `--no-llm` override per run.

Alternatives considered

- **A workflow framework (Airflow, Prefect)**: built for scheduled batch jobs, far too heavy for a six-step function call.
- **Load models inside `run`**: simpler, but every API request would pay seconds of loading.

Interview notes

- *What is the order of stages?* Ingest and clean, segment, classify, extract metadata and facts, evaluate the policy, explain, build the report. Metadata comes after classification because governing law is read from the classified governing-law clauses.
- *How fast is it (E8)?* Without the LLM, about 0.2 to 0.3 s per text PDF or DOCX and about 7 s per scanned contract (OCR dominates, about 3 s per page) on a CPU. The target was under 30 s for a 20-page text PDF.
- *What happens if Gemini is down?* Retries with backoff, then template explanations and a warning. Statuses do not change.

10 Backend API

Files: `backend/main.py`, `backend/api/contracts.py`, `backend/api/policies.py`, `backend/jobs.py`, `backend/tables.py`, `backend/db.py`, `backend/state.py`, `backend/errors.py`.

What

A FastAPI application under `/api/v1` that accepts uploads, runs analyses in the background one at a time, stores results in SQLite, and serves reports, clauses and policies to the UI.

Why

Analysis takes from under a second to over a minute (OCR, LLM calls), which is too long for one HTTP request. A job queue with status polling keeps the UI responsive. Keeping all NLP in the `contractguard` package and all HTTP concerns here means either side can change independently.

How

Startup (`main.py`)

`create_app` uses FastAPI's lifespan hook to build an `AppState`: settings, a SQLModel engine for `data/contractguard.db`, the uploads folder, the policies folder, and `Resources.load(...)` (models loaded once). If loading fails (for example no trained classifier), the API **still starts**: `/health` reports the reason and uploads return 503 (D43). CORS allows only the Streamlit origin (D45).

Endpoints

Method and path	Purpose	Notes
`GET /health`	readiness	models, embeddings, Tesseract, LLM provider, version, disclaimer
`POST /contracts`	upload	multipart: `file`, `contract_type`, `our_org_name`, `policy_id`, `use_llm`; returns 202 and `contract_id`
`GET /contracts`	list	newest first, with counts
`GET /contracts/{id}/status`	poll	status, stage, progress, error
`GET /contracts/{id}/report`	report	409 until DONE
`GET /contracts/{id}/clauses`	clauses	409 until DONE
`DELETE /contracts/{id}`	delete	removes rows and the uploaded file
`GET /policies`, `GET /policies/{id}`	read policies	
`PUT /policies/{id}`	edit a policy	schema-validated, versioned

Interactive documentation is generated automatically at `/docs`.

Upload checks (D44)

Before queueing: the extension is `.pdf` or `.docx` (415 otherwise), the contract type is known (422), the policy exists (422), the size is within `limits.max_file_mb` (413), the file is not empty (422). The filename keeps only letters, digits, `._ -`, the base name only, at most 120 characters. Magic bytes,

page and word limits are checked by the pipeline, so a renamed file is accepted, then FAILED with a readable error.

Jobs (`jobs.py`)

`run_job` runs in FastAPI's `BackgroundTasks` behind a `threading.Semaphore(1)`: one analysis at a time, to avoid CPU and memory spikes and to respect LLM rate limits. The progress callback writes the stage and percentage to the contract row. On success `store_report` saves the whole report as JSON plus one row per clause and per finding; on failure the row becomes FAILED with the error message (internal errors show only the exception type, never a stack trace).

Storage (`tables.py`)

Table	Holds
`contracts`	id, filename, type, status, stage, progress, error, options, policy version, timestamps, `report_json`, `counts_json`
`clauses`	one row per clause, for filtering
`findings`	one row per finding, with the explanation as JSON
`llm_cache`	cached LLM answers (shared with the explainer)

Errors (`errors.py`)

Every error response is `{"error_code": ..., "message": ...}`, including request validation errors (`VALIDATION_ERROR`, 422).

Callers and dependencies

Called by the Streamlit UI through `frontend/api_client.py`. Uses FastAPI, Uvicorn, SQLModel (SQLAlchemy), python-multipart and the `contractguard` pipeline.

Errors and edge cases

- A contract deleted while running: updates find no row and stop quietly.
- Models not loaded: 503 `MODELS_NOT_LOADED` with the reason.
- No authentication in V1 (single user on localhost); the public demo is shared by all visitors.

Configuration

`paths.sqlite_path`, `paths.uploads_dir`, `limits.*`, `frontend.origin`, plus everything the pipeline reads.

Alternatives considered

- **Celery and Redis for jobs**: robust but two more services; one in-process worker is enough for a single-user tool.
- **PostgreSQL**: unnecessary for one user; SQLite is a single file with no server.
- **Flask**: no built-in validation or OpenAPI docs; FastAPI generates both from type hints.

Interview notes

- *Why 202 Accepted?* The request is accepted for processing that has not finished; the client polls the status endpoint.

- *How do you protect against malicious uploads?* Extension and magic-byte checks, size, page and word limits, sanitised filenames, files stored under a random id, encrypted PDFs rejected.
- *Why store the report twice (JSON and rows)?* The JSON is what the UI shows in one read; the rows allow filtering and future queries across contracts.
- *How is it tested?* `TestClient` and a fake classifier with the mock LLM, including the failure paths (bad type, oversized file, unknown policy, models not loaded).

11 Streamlit UI

Files: `frontend/app.py`, `home.py`, `ui.py`, `api_client.py`, `frontend/pages/1_Upload.py`, `2_Report.py`, `3_Clauses.py`, `4_Policy.py`.

What

A web interface with five pages: Home, Upload & Analyze, Report Dashboard, Clause Explorer and Policy Editor. It talks to the backend over HTTP only and never imports the NLP code.

Why

Reviewers are not programmers. Streamlit gives a working, accessible interface in pure Python, which the HLD fixed as the UI technology for this project.

How

- **`app.py`** defines the navigation with `st.navigation`, so the sidebar shows readable names and short URLs (`/upload`, `/report`, `/clauses`, `/policy`) instead of file names (D46).
- **`api_client.py`** wraps `requests`: every call turns HTTP errors into an `APIError` with the backend's `error_code` and message, and a connection failure into "Cannot reach the backend at ...". The backend URL comes from `CG_API_URL` or `frontend.api_url`.
- **`ui.py`** holds shared pieces: page setup, the disclaimer banner on every page, status and severity badges, the contract picker, and evidence highlighting.

Page	What the reviewer does
Home	Reads what the tool does; sees whether the backend, models, embeddings, OCR and LLM are ready
Upload & Analyze	Chooses a file, contract type (or auto), organisation name, policy and the AI toggle; watches a progress bar with the current stage; sees recent analyses
Report Dashboard	Overview card (title, type, dates, governing law, parties and roles, OCR pages), counts, executive summary, warnings, clause checklist, JSON download, and findings filtered by status, severity and category; each finding expands to show expected and found, location, the evidence with the quoted sentence highlighted, and the explanation
Clause Explorer	Filters by clause kind; a table of clauses with categories, confidence, how each was classified and the location; selecting a row shows the full text and every category score
Policy Editor	Picks a policy; edits rules in a table or as raw JSON; saving validates against the schema and creates a new version

Accessibility. Status and severity badges always carry text, never colour alone (WCAG 1.4.1), and dark mode is styled.

Callers and dependencies

Run with `streamlit run frontend/app.py`. Depends on Streamlit, pandas and requests; reads only the settings from `contractguard.config`.

Data in and out

In: user actions. Out: HTTP calls to the backend; the report JSON is shown and offered for download.

Errors and edge cases

- Backend down: a clear message instead of a stack trace.
- Models not loaded: the home page shows the backend's reason and uploads are refused with it.
- Upload progress polls `/status` every `frontend.poll_seconds` (2 s) until DONE or FAILED.

Alternatives considered

- **React or Vue front end**: more control, but a second language and build toolchain for a single-developer project.
- **Gradio**: suited to single-function demos, weaker for multi-page dashboards and tables.

Interview notes

- *Why does the UI not import `contractguard` directly?* Separation: the UI could be replaced without touching the NLP, and the same API serves other clients.
- *How was the UI tested?* Every page runs headless with Streamlit's `AppTest` against a real Uvicorn backend in a thread (fake classifier, mock LLM), plus Playwright screenshots at desktop and phone widths checked by eye (D47).
- *How do reviewers trust a flag?* Every finding shows the exact clause sentence it is based on, with the location, and the explanation's quote is checked to be in that clause.

12 Training the classifier

Files: `training/inspect_labels.py`, `prepare_cuad.py`, `prepare_ledgar.py`, `train_classifier.py`, `evaluate.py`, `build_prototypes.py`, `common.py`; mapping in `config/label_map.yaml`; outputs in `models/` and `evaluation/results/`. Step-by-step instructions: `training/README.md`.

What

Offline scripts that turn two public datasets into the clause classifier, its per-category thresholds and the category prototypes that the pipeline loads at runtime.

Why

There is no labelled set of Indian contract clauses. Two public English datasets cover most of the needed categories: **LEDGAR** (provisions from SEC filings, 100 labels, one per provision; from the LexGLUE benchmark) and **CUAD** (510 contracts annotated by lawyers for 41 clause types). The 8 gold contracts are never used for training or tuning; they are the final exam.

How

Step	Script	Output
1	`inspect_labels.py`: prints every label of both datasets and checks every name in `label_map.yaml` exists	`docs/labels_ledgar.txt`, `docs/labels_cuad.txt`
2	`prepare_cuad.py`: the paragraph containing each mapped annotation is a positive; up to 5 unannotated paragraphs per contract are negatives; split by contract 80/10/10 (GroupShuffleSplit, seed 42)	`data/processed/cuad_{train,val,test}.jsonl`
3	`prepare_ledgar.py`: mapped labels become categories, the rest OTHER; OTHER downsampled to 3 times the largest class in train only (D40); official splits kept	`data/processed/ledgar_*.jsonl`
4	`train_classifier.py`: fit the vectoriser on train only, try 5 model families over small grids, choose by validation macro-F1, tune per-category thresholds	`models/classifier.joblib`, `models/thresholds.json`, comparison table
5	`evaluate.py`: test-split scores (E1), per source, confusion matrix	`classification_test.*`, `classification_confusion.png`
6	`build_prototypes.py`: mean MiniLM embedding of up to 200 training positives per category (SLA from hand-written examples); tune the fallback threshold on validation	`models/prototypes.npy`, `prototypes.json`

Label masks (D39). Each row gets, per category, 1 (positive), 0 (true negative) or -1 (unknown, because that row's dataset never labels that category). `MaskedOneVsRest` trains each category only on rows where it is known.

Features. TF-IDF on words and bigrams, `min_df=2`, `max_df=0.9`, sublinear term frequency, lowercase, stop words without negations.

Leakage control. The saved artifact is one scikit-learn `Pipeline(vectoriser, classifier)` fitted on the training split only; CUAD is split by contract, so paragraphs of one contract never appear in two splits.

Results (run on 2026-10-26)

- Train rows 21,180, validation rows 10,418, test rows 10,399.
- Selected: TF-IDF + calibrated linear SVM, C = 1. Validation macro-F1 0.857 at 0.5, 0.875 with tuned thresholds. Test macro-F1 **0.854**, micro-F1 0.869 (target 0.75).
- Fallback threshold 0.80 (D50).

Errors and edge cases

- If Hugging Face downloads are blocked, LEDGAR and MiniLM cannot be fetched; CUAD downloads from GitHub. The MiniLM comparison model is skipped when MiniLM is unavailable.
- The pickled model depends on the scikit-learn version; it is pinned for deployment.

Alternatives considered

- **Train on the gold contracts**: there are only 8 and they are the test set. Rejected.
- **Fine-tune Legal-BERT**: likely a few points better, but needs a GPU and slows inference; the linear model already exceeds the target.
- **Use only CUAD**: too few examples for several categories, and no confidentiality or payment labels.

Interview notes

- *What is LEDGAR?* About 80,000 contract provisions from SEC EDGAR filings, each labelled by its heading, used as a LexGLUE benchmark task.
- *What is CUAD?* The Contract Understanding Atticus Dataset: 510 commercial contracts with more than 13,000 lawyer annotations over 41 categories.
- *How did you handle class imbalance?* Balanced class weights, downsampling OTHER in training, per-category thresholds, and macro-F1 for selection so rare categories count equally.
- *Why macro-F1?* Micro-F1 is dominated by frequent categories such as governing law; macro-F1 shows whether rare ones such as renewal work.

13 Gold set and evaluation

Files: `evaluation/gold/` (contracts, `build_gold.py`, `GUIDELINES.md`, `ADJUDICATION.md`, `labeller_ai/`), `evaluation/eval_ingestion.py`, `eval_segmentation.py`, `eval_facts.py`, `eval_policy.py`, `run_e2e_eval.py`, `make_rating_sheet.py`, `label_agreement.py`; results in `evaluation/results/`.

What

A synthetic test set of 8 contracts with known answers, and scripts that measure each stage and the whole system against it (metrics E1 to E8 of the HLD).

Why

Without measured numbers, any claim about accuracy is a guess. Each phase of the build ended with an evaluation run against acceptance targets, and the results are committed so they can be checked.

How

The gold set

Contract	Type	Planted issues
GC01 IT support	Vendor	7-day convenience notice for the vendor only, uncapped liability, auto-renewal, 15-day payment
GC02 Master services	Vendor	fully compliant baseline
GC03 Facility management	Vendor	Word auto-numbering, no SLA, no indemnity, English law, 30 business days notice
GC04 SaaS	SaaS	auto-renewal, 15-day notice for the provider
GC05 Software licence	SaaS	uncapped liability, New York law, no SLA
GC06 Mutual NDA	NDA	compliant
GC07 One-way NDA	NDA	Roman-numeral ARTICLE headings, auto-renewal, Singapore law
GC08 Equipment rental	Other	no confidentiality clause

Each contract is written once as `contract.yaml`. `build_gold.py` generates a DOCX (python-docx, with a running header and "Page X of Y" footer), a text PDF (LibreOffice headless), a simulated scan (200 DPI, slight skew, noise, blur, JPEG quality 60, image-only PDF) and `gold.json` with clause boundaries, categories, facts and expected rule statuses (D6). All 24 files are reproducible.

Labelling status. The labels were drafted automatically from the YAML sources (labeller 0). A second AI labelling from the PDFs only (`labeller_ai/`, D51) agreed on 100% of rule statuses and 97.2% of clause categories; its 8 disagreements are listed in `ADJUDICATION.md` for the owner to decide. Neither is a human labelling, so the gold labels remain a draft.

Results

Metric	What	Target	Result
E1	Clause classification, test split	macro-F1 at least 0.75	0.854

Metric	What	Target	Result
E2	Segmentation boundaries	F1 at least 0.85	1.000 DOCX, 0.986 PDFs
E3	OCR	CER under 5%; text vs scan status agreement at least 90%	CER 1.10%; agreement 100%
E4	Facts and metadata (gold categories)	at least 85%	100%
E5	Policy violations, end to end	recall at least 85%, precision at least 80%	88.9% / 100%
E6	Semantic search	precision at 3 at least 70%	35.2% (recall at 3: 96.5%, MRR 0.909)
E7	LLM explanations	rating at least 4/5, no invented values	18/20 grounded, 0 invented numbers; AI rating 4.0, human rating pending
E8	Speed	under 30 s for a 20-page text PDF	0.2 to 0.3 s per text contract, about 7 s per scanned one

About E6. Most rules have exactly one correct clause, so precision at 3 cannot exceed about 33% for those rules even with a perfect ranking. Recall at 3 (96.5%) and MRR (0.909: the right clause is almost always ranked first) show the search works; the target is the wrong measure for this data (recommended change, not yet made).

Why the numbers are optimistic. The labels are drafts, the patterns were developed while looking at these contracts, the scans are simulated, and 8 contracts is a small sample. A trustworthy estimate needs human labels and contracts never seen during development.

Scripts

Script	Measures
`eval_ingestion.py`	every file loads; OCR character error rate (Levenshtein)
`eval_segmentation.py`	boundary P/R/F1 with fuzzy matching of each clause's first eight words
`eval_facts.py`	exact match of the 7 facts and the metadata
`eval_policy.py`	status confusion matrix; violation precision and recall
`run_e2e_eval.py`	full pipeline, LLM off: E3, E5, E6, E8
`make_rating_sheet.py`	runs with the LLM; writes a rating sheet for two raters and automatic checks (E7)
`label_agreement.py`	agreement and Cohen's kappa between two labellings

Interview notes

- *Precision vs recall here?* Recall of violations matters more (a missed uncapped liability is worse than a false alarm), so its target is higher; precision still matters for trust.
- *What is Cohen's kappa?* Agreement corrected for chance: (observed − expected) / (1 − expected).
- *Why simulate scans?* Reproducible and fast; the downside is optimism, so at least one real scan is recommended.
- *What would you do next?* Two human labellers with adjudication, real contracts (CUAD public contracts, anonymised Indian ones with permission), at least one real scan, and confidence intervals on every metric.

14 Tests and tooling

Files: `tests/` (21 test files, 308 tests), `tests/conftest.py`, `tests/fakes.py`, `pyproject.toml`; strategy in `docs/TESTING.md`.

What

Unit, integration and UI tests run with pytest, plus three static checks: ruff (lint), ruff format (formatting) and mypy (types).

Why

Contract review flags are only useful if they are right. Tests pin down every rule that was learned the hard way (the D23 page-break bug, OCR Roman numerals, "including but not limited to"), so a later change cannot silently undo it.

How

Area	Test files	Examples
Foundations	`test_config.py`, `test_models.py`	environment overrides, secret hiding, offset maps
Ingestion	`test_ingestion.py`	PDFs and DOCX built inside the test; magic bytes; encrypted PDF; fake OCR; one real Tesseract test, skipped if Tesseract is absent
Cleaning	`test_cleaner.py`	header removal, the D23 page-break regression, the "(15) days" join
Segmentation	`test_segmenter.py`	numbering styles, OCR Roman numerals, fallback, long-clause splits
Extraction	`test_durations.py`, `test_facts.py`	number words, business days, negation windows, minimum notice, ambiguity
Classification	`test_classifier.py`	thresholds, SLA rule, embedding fallback with a fake embedder, missing artifacts
Policy	`test_policy_engine.py`, `test_policy_schema.py`, `test_policy_file.py`	every status path, schema errors, versioned saves, the shipped policy is valid
LLM	`test_explainer.py`	grounding failures, invented numbers, retries with a fake clock, cache hits, templates
Pipeline and CLI	`test_pipeline.py`, `test_cli.py`	gold files end to end, warnings, exit codes
Backend	`test_backend.py`	every endpoint, bad uploads, models missing, policy versions
UI	`test_frontend.py`	every page headless with Streamlit `AppTest` against a real backend
Scripts	`test_build_gold.py`, `test_inspect_labels.py`, `test_label_agreement.py`	gold building and agreement maths

Isolation. `conftest.py` clears cached settings and patterns and removes environment overrides before each test. Slow or external pieces are injected: the PDF loader takes an `ocr_fn`, the explainer

takes a provider, a clock and a sleep function, and `fakes.py` supplies a fake classifier and embedder. The LLM is always mocked in tests.

Static checks. ruff rules E, W, F, I, B and UP with a 100-character line limit; mypy with the Pydantic plugin over `contractguard` and `backend`.

```
pytest                    # 308 passed (2026-11-08), also inside the Docker image on Python
3.12
ruff check . && ruff format --check .
mypy
```

Interview notes

- *Unit vs integration tests?* Unit tests check one function with controlled inputs; integration tests run several stages together, here the whole pipeline on the gold files and the API with the UI on top.
- *How do you test code that calls an LLM?* Dependency injection of a mock provider that can return scripted answers or raise errors, and an injected clock so retry waits take no time.
- *What did static typing catch?* 11 errors in Phase 0, such as values that could be `None` being used as strings.
- *What is a regression test?* A test added with a bug fix that reproduces the failing case, so the bug cannot come back unnoticed.

Testing strategy

How ContractGuard is tested, and what counts as evidence that a phase is done. Acceptance targets come from the HLD (§12, §14); this file says how each is checked.

Principles

- **The LLM is always mocked in tests** (`MockProvider`). No test calls a network service.
- **Deterministic.** No randomness at inference; generators use fixed seeds. The same input gives the same output, so tests compare exact values.
- **Behaviour, not implementation.** A test feeds realistic input (a clause, a PDF page) and checks the result a reviewer would see (a fact value, a status), not private helpers.
- **Every bug fixed gets a regression test** that fails before the fix.
- **The gold set is evaluation data, never test fixtures for tuning.** Unit tests use small inline strings or generated files under `tmp_path`.

Layers

Layer	Scope	Tooling	Where
Unit	One module: config loading, duration parsing, regex detectors, policy checks	pytest	`tests/test_<module>.py`
Component	One pipeline stage on real files: a generated DOCX/PDF through ingestion + cleaning	pytest + python-docx / PyMuPDF fixtures	`tests/test_<stage>_*.py`
Pipeline	`pipeline.run()` on gold contracts with the LLM off	pytest (marked `slow`)	`tests/test_pipeline.py`
API	Every FastAPI endpoint, success and error paths	`httpx` / FastAPI `TestClient`, temp SQLite	`tests/api/`
End to end	Upload → DONE → report in the UI	Playwright against local backend + Streamlit	`tests/e2e/` (Phase 9)
Evaluation	Metrics E1–E8 against targets	scripts in `training/` and `evaluation/`	`evaluation/results/`

Evaluation scripts are not pass/fail tests: they produce reports. A phase is accepted when its metric meets the HLD target in the generated report.

What every stage must cover

- **Normal case** on realistic text.
- **Edge cases** named in the HLD for that stage (e.g. negation within 8 tokens, "including without limitation", "thirty (30) business days", DOCX auto-numbering, PDFs under 3 pages).
- **Invalid input**: wrong file type, renamed file, encrypted PDF, empty document, oversize file.
- **Failure paths**: missing Tesseract, LLM timeout/429, ambiguous facts → `NEEDS_REVIEW`.
- **Security-sensitive paths**: filename sanitization, magic-byte checks, secrets never printed or logged, full contract text never logged at INFO.

Quality gates (run before every commit)

```
ruff format --check .
ruff check .
mypy                # pydantic plugin; config in pyproject.toml
pytest
```

All four must pass. Results are quoted verbatim in each phase summary.

Phase acceptance checks

Phase	Check	Target
0	gates above; `python -m contractguard.cli --help`	all green
1	`build_gold.py` validates all 8 sources and builds 24 files; two human labellings	24 files; agreement logged
2	every gold file → `Document`; OCR CER on scanned gold files	24/24; CER < 5%
3	`python evaluation/eval_segmentation.py`: boundary P/R/F1 on 10 annotated contracts (E2; 8 gold now, 2 CUAD pending)	F1 ≥ 0.85
4	macro-F1 on combined test set (E1)	≥ 0.75
5	`python evaluation/eval_facts.py`: fact exact-match accuracy on gold (E4); ≥ 25 duration/liability unit tests (`tests/test_durations.py`, `tests/test_facts.py`)	≥ 85%
6	violation recall/precision on gold, LLM off (E5); a test per check type and status branch	R ≥ 0.85, P ≥ 0.80
7	grounding pass rate; 30-finding rating sheet by 2 raters (E7)	mean ≥ 4.0, 0 hallucinated values
8	every endpoint tested; upload → DONE for text PDF, scanned PDF, DOCX	all pass
9	Playwright journeys at desktop and phone widths; accessibility checklist	all pass
10	E1–E8 report complete; NLP time for a 20-page PDF (E8)	< 30 s

Fixtures

- Small DOCX/PDF files are generated inside tests (python-docx, PyMuPDF) so no binary fixtures are committed for unit tests.
- Gold files under `evaluation/gold/` are used only by pipeline tests and evaluation scripts.
- Settings are isolated per test (`tests/conftest.py` clears caches and environment overrides).

Deployment

ContractGuard ships as one Docker image that runs the FastAPI backend on `127.0.0.1:8000` and the Streamlit UI on port 7860. The UI talks to the API inside the container, so only port 7860 is exposed. The public demo runs on a free Hugging Face Space (Docker SDK).

What is in the image

Part	Source
Ubuntu 24.04, Python 3.12, Tesseract 5 (English)	`apt`
Python packages, pinned to the tested versions	`deploy/requirements.txt`
PyTorch, CPU-only wheel	`download.pytorch.org/whl/cpu` (build argument `TORCH_INDEX_URL`)
spaCy `en_core_web_sm`	GitHub release wheel
MiniLM embedding model	downloaded at build time; if that fails, the app runs without semantic checks
Trained classifier, thresholds, prototypes	`models/` (committed; the classifier is xz-compressed to 8.5 MB, D53)
Code, config, policies	the repository

`deploy/start.sh` starts uvicorn, waits until `/api/v1/health` answers, then starts Streamlit. The container runs as user 1000, which Hugging Face Spaces requires.

Run it locally

```
docker build -t contractguard .
docker run -p 7860:7860 contractguard
# open http://localhost:7860
```

Optional AI explanations: add `-e LLM_PROVIDER=gemini -e LLM_MODEL=<model> -e GEMINI_API_KEY=<key>` to `docker run`, or point at a local Ollama with `-e LLM_PROVIDER=ollama -e LLM_MODEL=qwen2.5:3b -e CG_LLM__OLLAMA_HOST=http://host.docker.internal:11434`. Never put a key in the Dockerfile.

Hugging Face Space (automatic)

`.github/workflows/deploy-space.yml` uploads the files the image needs to the Space on every push to `main` (and on demand from the Actions tab). The Space then rebuilds the image itself.

One-time setup:

1. Create a free Hugging Face account and a Space: SDK **Docker**, template **Blank**, hardware **CPU basic**.
2. Create a Hugging Face access token with the **Write** role.
3. In the GitHub repository, **Settings → Secrets and variables → Actions**: add the secret `HF_TOKEN` (the token) and the variable `HF_SPACE` (`username/space-name`).
4. Optional, in the Space's **Settings → Variables and secrets**: the secret `GEMINI_API_KEY` and the variable `LLM_MODEL`. Without them, explanations use templates.

The upload uses `huggingface_hub.HfApi.upload_folder`, which stores binary files (the model and the PDFs) the way the Hub requires. The Space card comes from `deploy/huggingface/README.md`.

Limits of the free demo

- 2 vCPU and 16 GB RAM: a text PDF takes about a second; a scanned page takes a few seconds of OCR. Gemini calls are paced at 8 per minute (`llm.rpm`).
- No persistent disk: uploads, reports and edited policies are lost when the Space restarts or sleeps after inactivity.
- No login: every visitor sees the same contract list. Use synthetic or public contracts only.
- With Gemini on the free tier, Google may use the clauses sent to it.

Verified

The image was built and run in the development environment on 2026-11-08: `/health` reported models loaded and Tesseract available, the UI answered on its port, and the gold contracts GC01 (scanned PDF, 4 OCR pages, about 11 s) and GC03 (DOCX, under 1 s) produced the expected findings through the API, and all 308 tests passed inside the image on Python 3.12. MiniLM could not be downloaded there (network policy), so that run had no semantic checks; on Hugging Face the model downloads at build time.

Behind a TLS-inspecting proxy

Put the proxy's CA certificates as `*.crt` files in `deploy/certs/` (git-ignored) before building, and pass the proxy as build arguments (`--build-arg https_proxy=...`).

Demo script

A walkthrough for presenting ContractGuard. Use only synthetic or public contracts, because free-tier LLM data may be used by the provider.

Before the demo

```
source .venv/bin/activate
uvicorn backend.main:app --port 8000      # terminal 1
streamlit run frontend/app.py             # terminal 2, opens http://localhost:8501
```

Check the **Home** page: Backend Online, Models Loaded, OCR Available. If you have no API key, set `LLM_PROVIDER=none` in `.env`: every flag still gets a template explanation.

Files to use: `evaluation/gold/gc01/contract_scanned.pdf` (vendor contract with planted issues, scanned) and `evaluation/gold/gc02/contract.docx` (a mostly compliant one).

1. The problem (1 minute)

"Reviewing vendor contracts against an internal checklist is slow and repetitive. ContractGuard does the first pass and gives the reviewer a short list of flags, each with the rule, the evidence and the page. It does not give legal advice; the banner on every page says so."

2. Upload a scanned contract (2 minutes)

1. **Upload & Analyze** → choose `gc01/contract_scanned.pdf`, type *Auto-detect*, organization *ABC Technologies Pvt. Ltd.*, AI explanations on → **Analyze**.
2. Point at the progress stages: reading with OCR, finding clauses, classifying, checking the policy, writing explanations.

3. The report (4 minutes)

1. **Overview card**: detected type, both parties and which one is us, effective and expiry dates, governing law, OCR pages.
2. **KPI tiles** and the **executive summary**.
3. **Clause checklist**: which of the 9 topics were found.
4. Open **R03 Minimum termination notice**: expected ≥ 30 days, found 7 days, the sentence is highlighted inside clause 10, and the explanation is labelled *AI-generated* or *Template*. Say: "The rule engine decided this status. The AI only explains it, and its quote is checked against the clause text; if it invents a number or misquotes, we fall back to a template."
5. Filter by **Severity = HIGH**; download the JSON report.

4. Clause Explorer (1 minute)

Select a clause: full text, its kind, and the category scores as a bar chart. Show a clause marked *low confidence* if there is one (embedding fallback).

5. Change the policy (1 minute)

Policy Editor → set R03's threshold to 7 → **Save rules**: version goes 1.0.0 → 1.0.1 and the old version is kept in `policies/history/`. Re-analyse gc01: R03 is now compliant. Then try an invalid

value (severity `SEVERE` in the JSON tab) to show inline validation. Set it back.

6. Honest numbers (1 minute)

Open `evaluation/results/`: classifier macro-F1 (E1), policy recall and precision (E5), OCR error rate (E3), retrieval (E6), speed (E8). Say what makes them optimistic: synthetic gold contracts, simulated scans, draft labels, US training data for Indian contracts (README, Known limitations).

Likely questions

- *What if the LLM is down?* The report is produced anyway with template explanations and a warning; status and severity never depend on the LLM.
- *What leaves the machine?* Only the single clause behind each flagged finding, and only when AI explanations are on. Ollama keeps everything local.
- *Why not let the LLM decide compliance?* Determinism, auditability, and no hallucinated verdicts: the same file and policy always give the same findings.

Decision log

Choices not fixed by the HLD, recorded as required by HLD §0. Newest last.

#	Date	Phase	Decision	Reason
D1	2026-10-22	0	Packaging via `pyproject.toml` (setuptools) with extras `training`, `app`, `ollama`, `dev` instead of two requirements files.	One file; the core install stays free of FastAPI/Streamlit, matching HLD §0.2.
D2	2026-10-22	0	Env overrides: `CG_` prefix with `__` nesting (e.g. `CG_LLM__RPM=4`), plus the flat names `LLM_PROVIDER`, `LLM_MODEL`, `GEMINI_API_KEY` from `.env.example`. Precedence: env > `.env` > `settings.yaml`.	Lets the Windows Tesseract path and model name be set without editing YAML.
D3	2026-10-22	0	`llm.model` added to `settings.yaml` (default `null`, set via `LLM_MODEL`); `llm.provider` may also be `none`. `paths` section added for data/uploads/sqlite/models. `policy.policies_dir` added.	HLD §9.1 forbids hard-coding the model; paths must not be hard-coded either (§0.4).
D4	2026-10-22	0	Use `import pymupdf` (not `fitz`).	PyMuPDF ≥ 1.24 deprecates the `fitz` alias.
D5	2026-10-22	0	Tooling targets Python 3.11 (`requires-python >= 3.11`, ruff `py311`).	HLD §0.6.
D6	2026-10-22	1	Gold contracts are authored once as YAML (`evaluation/gold/src/*.yaml`) and `build_gold.py` generates the DOCX, the text PDF (LibreOffice headless export of the DOCX) and the scanned PDF (rendered at 200 DPI, seeded skew/noise/blur, image-only), plus `gold.json`.	Single source of truth; all 24 files are reproducible.
D7	2026-10-22	1	GC03 uses real Word auto-numbering (`w:numPr`) for its clause numbers; the other seven type numbers as text.	Exercises the DOCX numbering synthesis of HLD §6.1 on at least one gold file.
D8	2026-10-22	1	DOCX files carry a running header and a "Page X of Y" footer.	The exported PDFs then exercise header/footer and page-number removal (HLD §6.2).
D9	2026-10-25	1	Gold expiry dates use the legal reading: a term of N years from date D ends the day before the Nth anniversary (1 Apr 2026 + 1 year → 31 Mar 2027). The HLD's "effective date + duration in days" is an implementation shortcut; Phase 5 must match the legal reading.	Sanjeev asked for the best option (2026-10-25); a reviewer expects the legal date.
D10	2026-10-25	1	GC06 and GC07 render to 2 pages, so header/footer removal (needs ≥ 3 pages, HLD §6.2) is skipped for them. Kept as is.	Exercises the short-document branch; their running header stays in the text.
D11	2026-10-25	1	Automatically drafted gold labels are "labeller 0". Sanjeev is labeller 1; labeller 2 is still to be named.	HLD §12.2 requires two independent human labellers.
D12	2026-10-25	2	`Block` gains an optional `style` field (DOCX paragraph style); new stage models `TextSpan` and `CleanedText` carry the cleaned text and its offset map.	HLD §6.1 keeps `Heading*` styles as segmentation hints and §6.2 requires the offset map, but §5 has no field for either. Additive, optional fields only.

#	Date	Phase	Decision	Reason
D13	2026-10-25	2	Cleaning adds one rule to HLD §6.2 step 4: a line holding only a clause number ("10.1") is joined with the next line. Pattern `cleaning.number_only_line`.	LibreOffice-exported PDFs of Word auto-numbered lists put the number on its own line; without the join, segmentation would miss sub-clauses.
D14	2026-10-25	2	Tesseract missing → scanned pages raise `OCR_UNAVAILABLE` for the whole PDF (no partial report).	HLD §6.1: "scanned PDFs return a clear error".
D15	2026-10-25	2	Known limitation of the HLD header/footer rule: on very short pages every line sits in the top/bottom zone, so body lines that differ only in digits can be dropped as "repeating". Kept as specified; real headers are the target.	Found by a unit test; documented rather than diverging from the HLD.
D16	2026-10-25	2	The scanned gold PDFs are simulated (render + skew + blur + noise), so the measured OCR CER (1.1% mean) is likely optimistic versus real print-and-scan. A real scan of at least one contract is recommended before the final report.	Honest reporting of E3.
D17	2026-10-25	3	OCR-tolerant roman numerals: after `ARTICLE`, the characters `l`, `1`, `i` and `|` are read as `I` ("ARTICLE Ill" → III, "ARTICLE	" → I, "ARTICLE Vil" → VII).
D18	2026-10-25	3	A clause-start line has a heading only if it is all caps, or ≤ 12 words, does not end in `.` / `;` / `,`, and ≥ 60% of its content words are capitalised. All-caps headings are shown in title case.	The HLD's "short line" test took wrapped sentences ("10.3 On termination, the Contractor shall withdraw its") as headings.
D19	2026-10-25	3	A short clause merged into the next keeps the next clause's number and takes the short clause's heading ("3. TERM" + "3.1 ..." → number 3.1, heading Term, path `3 > 3.1`).	Matches how the gold set records sub-clauses and keeps the parent heading visible.
D20	2026-10-25	3	Segmentation is scored by matching each gold clause's first eight words to a predicted clause's opening text (rapidfuzz partial ratio ≥ 85, one to one, in order, look-ahead 3), micro-averaged.	Robust to OCR slips; offsets differ across formats so character-exact boundaries cannot be compared.
D21	2026-10-25	3	The segmentation score (F1 0.986–1.000) is optimistic: the gold contracts were generated with clean, regular numbering, and the labels are still a draft. The HLD asks for 10 contracts; the 2 CUAD contracts are pending Hugging Face access. The only misses are the GC06/GC07 PDF preambles, where the running header left by D10 precedes the title.	Honest reporting of E2.
D22	2026-10-25	5	Termination notice: a duration introduced by "within" is skipped (it is a cure period, as in "fails to cure within 30 days of notice"). Otherwise HLD 7.3 as written: 12-token window to "notice", convenience sentences preferred, minimum taken.	Without it, GC08 returned the 30-day non-payment cure period instead of the 60-day notice.
D23	2026-10-25	5	Header/footer removal (Phase 2) now counts top-zone and bottom-zone repeats separately.	A sentence wrapping across a page break ("fifteen" / "(15) days' notice" at the foot of page 2, "(30) days' notice" at the head of page 3) was being deleted as a running header in GC04's PDFs. Found by the Phase 5 evaluation.

#	Date	Phase	Decision	Reason
D24	2026-10-25	5	Soft-wrap joining also joins a line that starts with "(15) days"-style text (a parenthesised number then a lowercase word) to the previous line.	"fifteen\n(15) days' notice" was split, so the notice sentence lost its duration. List items such as "(1) The Vendor" start uppercase and are not joined.
D25	2026-10-25	5	Fact evidence is the whole sentence that matched (a verbatim substring of the clause), not only the matched span. Conflicting values give `ambiguous=True` with value None (except termination notice: minimum). Facts whose category has no clause are None with no evidence.	Phase 7 explanations must quote evidence; Phase 6 then sends ambiguous facts to NEEDS_REVIEW.
D26	2026-10-25	5	`counterparty_unilateral`: False if any notice sentence is mutual or gives the right to our organization; True only if every convenience right belongs to the counterparty; None when roles are unknown or there is no convenience wording.	HLD 7.3 gives the idea; this fixes the edge cases (both parties with separate rights, as in GC04).
D27	2026-10-25	5	Sentences: spaCy sentencizer plus a merge after abbreviations listed in `patterns.yaml` (`Pvt.`, `Ltd.`, `Rs.`, ...). The spaCy EntityRuler for money and durations (HLD 7.1) is not built: no V1 fact uses money entities, and durations are matched by `durations.py`. `en_core_web_sm` is used only for the ORG fallback for parties.	Keep only what a fact needs; documented as a deliberate gap.
D28	2026-10-25	5	`extraction/document.py` (not in the HLD layout) assembles `ContractMeta`. The expiry date is computed only when the term sentence mentions the Effective Date (GC08's "12 months from delivery" gives None, as in gold).	One place for document-level fields.
D29	2026-10-25	5	E4 is measured with gold clause categories (no classifier yet): 100% on all 3 formats, 56/56 facts per format, plus metadata 100%. This measures the extractors on correctly classified clauses on contracts the extractors were developed against; the end-to-end figure (with the Phase 4 classifier) and labels confirmed by humans will be lower and are what the final report should quote.	Honest reporting of E4.
D30	2026-10-25	6	All deterministic finding text (expected, found, reasons, category labels, template explanations) lives in `config/templates.yaml`, loaded by `get_templates()`.	HLD 0.4: no hard-coded wording in Python; reviewers can reword without code changes.
D31	2026-10-25	6	Findings with the same status and severity are ordered by the reading position of their first evidence clause, then rule id (HLD 8.3 says "then location").	Sorting the location strings would put "Page 10" before "Page 2" and mix PDF pages with DOCX sections.
D32	2026-10-25	6	The embedder is optional in the engine. When MiniLM cannot be loaded, step 3's semantic search is skipped (an empty category goes straight to MISSING / NOT_APPLICABLE) and the pipeline adds a warning.	HLD 16 robustness: a failed stage gives a partial report, not a crash.

#	Date	Phase	Decision	Reason
D33	2026-10-25	6	Evidence and location come from the fact's `clause_ids` when it has them, else from all matched clauses. A fact whose value type does not fit the check (e.g. a string for NUMERIC_MIN) gives NEEDS_REVIEW. A saved policy's new version is always the stored version with the patch bumped, whatever version the client sent; duplicate rule ids are rejected.	Points the reviewer at the sentence that decided the status; never crash or silently pass on a bad fact or edit.
D34	2026-10-25	6	E5 measured with gold clause categories and embeddings off: violation recall and precision 100% (18/18) on all 3 formats, exact status accuracy 100% (80/80 per format). Optimistic for the same reasons as D29; the end-to-end figure with the classifier is the one to quote.	Honest reporting of E5.
D35	2026-10-25	7	`OllamaProvider` calls Ollama's HTTP API (`POST /api/chat`, `format` = JSON schema) with the standard library; the `ollama` extra was removed. Host is `llm.ollama_host` (new setting).	One less dependency; the HTTP API is stable and easy to mock.
D36	2026-10-25	7	The `llm_cache` table is written with stdlib `sqlite3` from the core package, in the same database file the backend uses. Without a path it is in memory.	HLD 0.2 keeps the core free of web/ORM imports; the CLI still caches.
D37	2026-10-25	7	Besides the HLD quote-grounding check, every number in an LLM answer (except the quote) must appear in the prompt inputs; otherwise the answer is treated like a grounding failure (one retry, then template). The executive summary gets the same check, and its prompt includes the status counts.	E7 targets 0 hallucinated values; this enforces it mechanically instead of only measuring it.
D38	2026-10-25	7	Gemini uses `response_json_schema` with `response_mime_type=application/json`; a stray ```json fence is still tolerated. Retryable: 408, 429, 5xx and network errors. Untested against the live API in this environment (no API key yet).	Reliable JSON; honest note that only mocked tests ran.
D39	2026-10-25	4	Multi-label training uses label masks: a row's label for a category is 1 (positive), 0 (true negative) or -1 (unknown, skipped for that category). LEDGAR rows are negatives for every LEDGAR-mapped category; CUAD rows are negatives only for `cuad_negatives` (LIABILITY, GOVERNING_LAW, RENEWAL). `MaskedOneVsRest` replaces `OneVsRestClassifier`; scores are masked the same way.	CUAD has no confidentiality / payment / indemnity labels, and its termination and IP labels are narrow (convenience termination, IP assignment), so treating unlabelled CUAD paragraphs as negatives would teach the model wrong answers. A CUAD-only smoke run confirmed it: without LEDGAR negatives, TERMINATION and IP are predicted for everything.
D40	2026-10-25	4	LEDGAR OTHER is downsampled (to 3x the largest positive class) in the train split only; validation and test keep their natural distribution. Categories with no training positives are dropped from the model with a warning.	Test scores then reflect real class balance; a model never claims a category it has no data for.
D41	2026-10-25	4	`Report` gains optional `file_type`, `num_pages`, `ocr_pages`, `llm_used`, `timings_s`. Counts: HIGH/MEDIUM/LOW count NON_COMPLIANT + MISSING findings by severity; each status has its own count; `CLAUSES` counts BODY + EXHIBIT clauses.	HLD 11 overview card needs file type and OCR pages; E8 needs per-stage timings.

#	Date	Phase	Decision	Reason
D42	2026-10-25	4	Clauses the classifier leaves empty are first checked by the SLA keyword rule, then the prototype fallback; an SLA keyword match on a clause that also has classifier categories adds SLA without changing `classified_by`. Missing classifier artifacts stop the run with `MODEL_ARTIFACT_ERROR`; missing prototypes only disable the fallback. The embedding model is loaded from the local cache first.	HLD 6.4 order; a run without a classifier would report every rule as MISSING, which is worse than a clear error.
D43	2026-10-25	8	If models fail to load at startup, the API still starts: `/health` reports `models_loaded=false` with the reason and uploads return 503 `MODELS_NOT_LOADED`. `/health` also reports `embeddings_loaded`.	A clear message in the UI beats a backend that will not start.
D44	2026-10-25	8	Upload checks before queueing: extension (.pdf/.docx), contract type, policy exists, size limit, non-empty. Magic bytes, page and word limits are checked by the pipeline's router, so a renamed file is accepted as QUEUED and then FAILED with a readable error. Filenames keep only `[A-Za-z0-9._ -]`, base name only, max 120 characters. `GET /contracts/{id}/clauses` also returns 409 until DONE. Report JSON is served as stored.	One place for file validation (the router), and the stored report is returned byte for byte.
D45	2026-10-25	8	CORS allows only `frontend.origin` (new setting, default `http://localhost:8501`).	HLD 10.2 "Streamlit origin only"; `frontend.api_url` is the backend's address, not the UI's.
D46	2026-10-25	9	The UI entry `frontend/app.py` defines `st.navigation` with readable titles and short URLs (`/upload`, `/report`, `/clauses`, `/policy`); the home page is `frontend/home.py`. The backend URL can be overridden with `CG_API_URL`. Severity and status badges always carry text, never colour alone; dark mode is styled.	Sidebar showed file names ("app", "2_Report"); accessibility (WCAG 1.4.1).
D47	2026-10-25	9	UI tests run every page headless with Streamlit `AppTest` against a real uvicorn backend in a thread (fake classifier, mock LLM), plus Playwright screenshots at desktop and phone widths checked by eye.	HLD 14 "the 4 pages work end-to-end against the local backend", tested the way the UI really talks to it.
D48	2026-10-25	10	`run_e2e_eval.py` measures E3 as rule-status agreement between each contract's text PDF and scanned PDF; E6 counts a retrieved BODY clause as relevant when it lies inside one of the rule's gold clauses (same number or a sub-clause); rules with no gold clause (MISSING, NOT_APPLICABLE) are not retrieval queries. E7's automatic "grounding pass rate" is the share of flagged findings explained by the LLM (passed both checks).	Concrete definitions for HLD 12.1 metrics that the HLD names but does not define.
D49	2026-10-25	1/4	`inspect_labels.py` reads CUAD from the local `CUADv1.json` when present (GitHub download) and reports a dataset it cannot load instead of crashing. CUAD names verified (41 categories, `docs/labels_cuad.txt`). LEDGAR names are confirmed only against the LexGLUE dataset card so far, so `label_map.yaml` stays `verified: false`.	Hugging Face file downloads are blocked in the build environment; GitHub is not.

#	Date	Phase	Decision	Reason
D50	2026-10-26	4	`label_map.yaml` set to `verified: true`: `inspect_labels.py` printed all 100 LEDGAR labels from the dataset itself and every mapped name exists. `embeddings.proto_threshold` = 0.80 from `build_prototypes.py`. Fallback accuracy rose monotonically over the grid (0.03 at 0.30 to 0.98 at 0.80) because 8,657 of the validation rows the classifier leaves unlabelled are OTHER, so the tuned value is the top of the grid and the fallback rarely fires.	Hugging Face downloads work in the new environment. The threshold follows the documented tuning rule; widening the grid or changing the metric is left as a follow-up rather than tuned on the gold set.
D51	2026-10-26	1/10	An AI assistant labelled GC01–GC08 a second time from `contract_text.pdf` only (`evaluation/gold/labeller_ai/`), at the project owner's request, and `label_agreement.py` compares it with the draft. Disagreements go into `ADJUDICATION.md` as pending; the gold labels were not changed.	A consistency check of the draft. It is not independent human labelling (both labellings are automated), so labellers 1 and 2 are still needed.
D52	2026-10-26	10	E7 runs on a local model: Ollama 0.12.6 (GitHub release) serving Qwen2.5-3B-Instruct Q4_K_M, loaded from the Hugging Face GGUF with a ChatML template in a Modelfile, `LLM_PROVIDER=ollama LLM_MODEL=qwen2.5:3b CG_LLM__TIMEOUT_S=180`.	No Gemini key, and ollama.com / registry.ollama.ai are blocked in the build environment. A small (3B) model keeps CPU-only generation practical on a 4-core machine; larger models were not tried.
D53	2026-11-08	11	Deployment: one Docker image (Ubuntu 24.04, Python 3.12 from apt, Tesseract) runs uvicorn on localhost and Streamlit on port 7860; a GitHub Action uploads it to a Hugging Face Space with `upload_folder`. Trained artifacts are committed under `models/`; the classifier is re-saved with xz compression (25 MB to 8.5 MB, identical predictions) to stay under the Hub's 10 MB plain-file limit. Runtime packages are pinned in `deploy/requirements.txt`.	The owner chose Hugging Face Spaces (free, 16 GB RAM). Ubuntu instead of Debian because only Ubuntu mirrors were reachable where the image was built and tested; one container keeps the free single-port Space simple.